

Studi'OS Desktop

Roadmap - Application desktop, daemon, Knowledge Vault, Graphify et visualiseurs natifs

Chantier client visant une installation sans ligne de commande et un environnement Studi'OS homogène sur chaque machine.

Vision : l'utilisateur installe Studi'OS comme un logiciel classique. L'application contient le Dashboard, gère le daemon local en arrière-plan, connecte la machine au serveur, fournit un Knowledge Vault Markdown, l'intégration Graphify/code graph et des visualiseurs natifs. Les outils externes (OpenCode, Claude Code, Obsidian, IDE...) restent optionnels.

Principe	Décision
Installation	Aucune commande requise pour le parcours normal : installateur Windows puis interface graphique.
Serveur	Le backend/API/MCP/DB reste distant ; aucun LLM n'est embarqué dans Studi'OS Desktop.
Desktop	Le Dashboard web existant est réutilisé dans une enveloppe desktop, avec capacités locales additionnelles.
Vault	Le format canonique est Markdown/fichiers locaux ; Obsidian est une intégration optionnelle, pas une dépendance.
Graphify	Moteur/code graph intégré ou distribué avec le client selon audit licence/packaging ; API interne isolée.
Graphes	Knowledge Graph et Code Graph restent des sources distinctes ; une vue Project Graph peut les projeter ensemble.
Agnosticisme IA	Les adapters configurent les harness détectés sans introduire leurs concepts dans le domaine serveur.

1. Résultat attendu

- R1 - Un utilisateur Windows installe Studi'OS et termine l'onboarding sans terminal.
- R2 - Studi'OS Desktop affiche le Dashboard classique dans une application native.
- R3 - Le daemon démarre, s'arrête et se met à jour avec l'application sans intervention manuelle.
- R4 - La machine peut être enregistrée/authentifiée graphiquement auprès d'un serveur Studi'OS.
- R5 - Un dossier local peut être associé à un projet Studi'OS via un sélecteur de fichiers.
- R6 - Le Git Watcher, heartbeat et services locaux ont un état visible et diagnostiquable.
- R7 - Chaque workspace peut disposer d'un Knowledge Vault Markdown intégré et optionnel.
- R8 - Le code graph/Graphify est indexable et consultable depuis l'application.
- R9 - Knowledge Graph et Code Graph disposent de visualiseurs natifs interactifs.
- R10 - Une vue Project Graph peut relier de façon projetée knowledge, code, tasks, roadmap, decisions et Git.
- R11 - Les harness IA présents sur la machine sont détectés/configurables via adapters, sans les rendre obligatoires.
- R12 - Le Dashboard web distant continue de fonctionner indépendamment de Studi'OS Desktop.

Invariants

- Le projet doit rester utilisable sans Desktop : le serveur et le Dashboard web conservent leur autonomie.
- Le Desktop ne doit pas devenir un second backend métier ; les données partagées restent canoniques côté serveur.
- Les données locales privées (vault, index code, chemins locaux) ne sont jamais envoyées au serveur par défaut.
- Aucun token/API key de fournisseur IA ne doit être requis par Studi'OS Desktop lui-même.
- Le daemon doit pouvoir survivre à la fermeture de la fenêtre si l'utilisateur choisit ce comportement.
- Knowledge Vault, Graphify/code graph et visualiseurs doivent pouvoir être désactivés par projet.
- Les formats locaux importants doivent rester documentés et récupérables sans l'application.
- Le packaging d'un composant tiers est conditionné à une vérification de licence et de redistribution.

2. Stratégie de parallélisation

P0-P2 stabilisent l'architecture desktop et les contrats locaux. Après P2, le chantier se divise en lanes indépendantes. La session d'intégration est la seule à modifier les frontières communes lors de la convergence.

Lane	Responsabilité	Phases
A - Shell	Desktop shell, navigation, lifecycle	P3
B - Daemon	Process manager, tray, heartbeat, watcher	P4
C - Workspace	Association dossiers/projets, config locale	P5
D - Knowledge	Vault Markdown + Knowledge Graph	P6
E - Code	Graphify/code graph + indexation	P7
F - Visualizers	Graph UI + Project Graph	P8
G - Adapters	Détection/config harness	P9
H - Packaging	Installer/update/security	P10
Integration	Onboarding, réconciliation, E2E	P11-P12

Principe : les lanes peuvent utiliser mocks/fixtures après P2. Elles ne doivent pas redéfinir les contrats locaux ou le protocole Desktop <-> daemon de leur côté.

P0 - Architecture Gate & audit

Objectif : Choisir l'architecture desktop et auditer l'existant avant tout packaging.

Dépendances : Aucune.

Parallélisation : Une seule session.

P0.1 - Auditer Dashboard Vite, daemon, Git Watcher, heartbeat, MCP local, Vault/Graphify adapters et mécanismes de provisioning existants.

P0.2 - Comparer Tauri/Electron selon besoins réels : sidecars, tray, filesystem, updater, signature, Windows packaging et frontend existant.

P0.3 - Auditer Graphify : licence, mode d'exécution, dépendances, poids, mise à jour et droits de redistribution.

P0.4 - Auditer le Vault actuel et séparer clairement format Markdown, indexation et dépendance éventuelle à Obsidian.

P0.5 - Définir les frontières Desktop / daemon / serveur / Dashboard web.

P0.6 - Définir menace locale : secrets, tokens machine, permissions filesystem, logs et processus enfants.

P0.7 - Définir compatibilité/version entre Desktop, daemon et serveur.

P0.8 - Produire DEC/ADR et matrice REOUIREMENT -> COMPONENT -> OWNER -> TEST.

Gate : aucune implémentation desktop avant validation de P0.

P1 - Contrats locaux & architecture client

Objectif : Figer les interfaces permettant le travail parallèle.

Dépendances : P0 accepté.

Parallélisation : Une session propriétaire.

P1.1 - Définir le protocole interne Desktop <-> daemon : status, start/stop, diagnostics, events et version.

P1.2 - Définir LocalWorkspaceConfig : project_id, chemin, repo, features locales activées ; aucun secret committable.

P1.3 - Définir le stockage sécurisé des credentials/tokens machine selon capacités OS.

P1.4 - Définir interfaces KnowledgeProvider, CodeGraphProvider et HarnessAdapter.

P1.5 - Définir un schéma commun de graphe de visualisation : nodes, edges, provenance, source et URI locale.

P1.6 - Définir politique de synchronisation : quelles métadonnées restent locales et lesquelles peuvent être publiées.

P1.7 - Définir erreurs/états : unavailable, indexing, stale, permission_denied, incompatible, disabled.

P1.8 - Créer fixtures contractuelles utilisées par toutes les lanes.

P2 - Socle Desktop minimal

Objectif : Créer une application desktop vide mais installable, connectée au Dashboard existant.

Dépendances : P1.

Parallélisation : Une session ; bloque la vague parallèle.

P2.1 - Créer le workspace/package Desktop sans déplacer la logique métier du Dashboard.

P2.2 - Embarquer/servir le build Dashboard dans la fenêtre desktop.

P2.3 - Ajouter bridge local minimal strictement typé pour capacités desktop.

P2.4 - Gérer ouverture/fermeture, liens externes et navigation sûre.

P2.5 - Ajouter identification/version Desktop visible dans Paramètres.

P2.6 - Créer build Windows de développement reproductible.

P2.7 - Tests smoke Desktop + non-régression Dashboard web.

Gate P2 : baseline commune pour toutes les lanes suivantes.

P3 - Desktop Shell & UX native

Objectif : Transformer le Dashboard en application desktop cohérente sans dupliquer son UI.

Dépendances : P2.

Parallélisation : Lane A.

P3.1 - Ajouter états desktop : connecté, hors ligne, daemon actif/inactif, version.

P3.2 - Ajouter intégration sélecteur dossier/fichier pour opérations locales autorisées.

P3.3 - Ajouter zone Paramètres > Application locale.

P3.4 - Ajouter raccourcis vers logs/diagnostics sans exposer de terminal.

P3.5 - Gérer reprise de session et erreurs réseau serveur.

P3.6 - Conserver responsive et fonctionnement navigateur sans APIs desktop.

P4 - Daemon lifecycle & services locaux

Objectif : Faire du daemon un composant invisible mais observable du logiciel.

Dépendances : P2.

Parallélisation : Lane B.

P4.1 - Packager le daemon comme sidecar/service contrôlé par Desktop.

P4.2 - Démarrage automatique configurable et arrêt propre.

P4.3 - Intégrer heartbeat et Git Watcher existants sans réimplémentation.

P4.4 - Ajouter tray Windows : état, ouvrir Studi'OS, pause services, quitter.

P4.5 - Ajouter health/status détaillé consommable par Desktop.

P4.6 - Gérer crash/restart borné et éviter les boucles de relance.

P4.7 - Logs locaux rotatifs et diagnostics exportables.

P4.8 - Tests lifecycle : fermeture UI, reboot simulé, crash daemon, réseau absent.

P5 - Workspace Manager & création de projet

Objectif : Relier proprement projets serveur et dossiers locaux sans ligne de commande.

Dépendances : P2.

Parallélisation : Lane C.

P5.1 - Ajouter + Nouveau projet dans l'expérience Dashboard/Desktop.

P5.2 - Parcours : nouveau projet, projet serveur existant, dossier local existant, dépôt Git existant.

P5.3 - Sélecteur natif de dossier et détection .git.

P5.4 - Créer/mettre à jour la config locale .studio sans y stocker de secrets.

P5.5 - Associer/dissocier un workspace sans supprimer les données projet.

P5.6 - Afficher repo/branche/watch status et chemin local dans le détail projet Desktop.

P5.7 - Supporter un projet sans dossier local et un dossier local sans Roadmap.

P5.8 - Tests multi-workspaces et chemins déplacés/indisponibles.

P6 - Knowledge Vault natif

Objectif : Fournir une mémoire Markdown locale standard à tous les utilisateurs.

Dépendances : P1-P2.

Parallélisation : Lane D.

P6.1 - Définir structure de vault par workspace et stratégie d'initialisation non destructive.

P6.2 - Implémenter lecture/recherche/indexation Markdown locale via KnowledgeProvider.

P6.3 - Ajouter navigateur Knowledge dans Desktop.

P6.4 - Ajouter éditeur Markdown léger uniquement si nécessaire ; préserver édition externe.

P6.5 - Indexer liens explicites, références et métadonnées pour produire Knowledge Graph.

P6.6 - Détecter éventuellement Obsidian et proposer Ouvrir dans Obsidian sans dépendance.

P6.7 - Brancher la source locale aux outils/contextes Studi'OS existants sans upload automatique.

P6.8 - Tests gros vault, fichiers invalides, liens cassés, renommage et feature désactivée.

P7 - Graphify / Code Graph intégré

Objectif : Standardiser l'intelligence structurelle du code côté client.

Dépendances : P0 licence validé, P1-P2.

Parallélisation : Lane E.

P7.1 - Choisir intégration : binaire sidecar, package embarqué ou installation gérée selon audit P0.

P7.2 - Créer CodeGraphProvider isolant Studi'OS des détails Graphify.

P7.3 - Indexation initiale et incrémentale d'un workspace.

P7.4 - Relier Git Watcher aux invalidations/réindexations sans déclencher des scans complets inutiles.

P7.5 - Exposer fichiers, symboles, relations calls/imports/contains selon capacités réelles.

P7.6 - État indexing/stale/error visible dans Desktop.

P7.7 - Préserver possibilité future de remplacer Graphify par un autre provider.

P7.8 - Tests performance, repo volumineux, langages non supportés et index corrompu.

P8 - Visualiseurs de graphes

Objectif : Rendre Knowledge Graph et Code Graph directement explorables dans Studi'OS.

Dépendances : P1 ; données P6/P7 mockables.

Parallélisation : Lane F.

P8.1 - Créer composant Graph Viewer commun : zoom, pan, sélection, recherche, filtres, légende.

P8.2 - Vue Knowledge Graph avec ouverture du document/note.

P8.3 - Vue Code Graph avec fichiers/classes/fonctions et relations disponibles.

P8.4 - Panneau détail d'un nœud avec provenance et relations.

P8.5 - Support grands graphes : limites, clustering/lazy expansion plutôt que tout afficher.

P8.6 - Créer Project Graph comme projection optionnelle, pas comme nouvelle source de vérité.

P8.7 - Relier progressivement code/knowledge aux Tasks, Roadmap, Decisions et Git quand références fiables existent.

P8.8 - Tests accessibilité, performances et graphes vides/partiels.

P9 - Harness detection & adapters

Objectif : Configurer les IA présentes sans imposer un fournisseur.

Dépendances : P1-P2.

Parallélisation : Lane G.

P9.1 - Détecter les harness supportés via adapters, sans scan excessif de la machine.

P9.2 - Afficher détecté/configuré/incompatible/non détecté.

P9.3 - Permettre configuration MCP Studi'OS par bouton avec preview des fichiers modifiés.

P9.4 - Sauvegarder/rollback la configuration locale lorsque possible.

P9.5 - Ne jamais demander les credentials fournisseur si Studi'OS n'en a pas besoin.

P9.6 - Prévoir registre d'adapters extensible sans modifier le core.

P9.7 - Tests OpenCode/Claude Code actuels + harness absent/inconnu.

P10 - Installer, mises à jour & sécurité Windows

Objectif : Distribuer Studi'OS comme un logiciel standard.

Dépendances : P2 ; lanes techniques suffisamment stables.

Parallélisation : Lane H.

P10.1 - Produire StudiOS-Setup.exe et installation/désinstallation propre.

P10.2 - Choisir emplacement données/config/cache/logs selon conventions Windows.

P10.3 - Signature de code/release process si infrastructure disponible ; documenter le chemin sinon.

P10.4 - Auto-update Desktop avec rollback/échec sûr.

P10.5 - Versionner/mettre à jour daemon et composants embarqués de manière compatible.

P10.6 - Préserver vaults/workspaces lors des upgrades et désinstallation par défaut.

P10.7 - Durcir bridge desktop, navigation, filesystem allowlist et lancement de sidecars.

P10.8 - Tests installation propre, upgrade, downgrade supporté, désinstallation et réinstallation.

P11 - Onboarding graphique & diagnostics

Objectif : Rendre l'installation réellement sans CLI.

Dépendances : P3-P5, P9-P10.

Parallélisation : Session d'intégration UI.

P11.1 - Premier lancement : URL serveur, connexion/authentification et enregistrement machine.

P11.2 - Vérifier daemon et services locaux automatiquement.

P11.3 - Proposer association/création du premier projet sans l'imposer.

P11.4 - Proposer activation Knowledge Vault/Code Graph avec explication simple.

P11.5 - Détecter les harness et proposer leur configuration, toujours optionnelle.

P11.6 - Créer écran Diagnostic/Doctor graphique : serveur, daemon, watcher, vault, graph, adapters.

P11.7 - Permettre réparer/retry les problèmes courants depuis l'UI.

P11.8 - Garantir qu'un utilisateur peut ignorer les fonctions avancées et utiliser seulement le Dashboard.

P12 - Réconciliation, dogfood & E2E final

Objectif : Assembler toutes les lanes et tester une installation réelle de bout en bout.

Dépendances : P3-P11.

Parallélisation : Une session intégratrice ; tests parallèles possibles.

P12.1 - Réconcilier branches sur baseline unique et vérifier contrats P1.

P12.2 - Build release sur machine propre/VM Windows.

P12.3 - E2E : installer -> lancer -> connecter serveur -> enregistrer machine -> ouvrir Dashboard.

P12.4 - Créer/associer projet -> dossier local -> Git Watcher/heartbeat visibles.

P12.5 - Activer Vault -> ajouter notes -> rechercher -> visualiser Knowledge Graph.

P12.6 - Indexer repo -> visualiser Code Graph -> suivre symbole vers fichier.

P12.7 - Afficher Project Graph avec relations disponibles sans inventer de liens.

P12.8 - Détecter un harness -> preview config -> appliquer -> vérifier MCP Studi'OS.

P12.9 - Fermer fenêtre -> vérifier comportement daemon/tray choisi -> rouvrir.

P12.10 - Tester serveur indisponible, daemon crash, index corrompu, vault absent, upgrade et désinstallation.

P12.11 - Vérifier Dashboard web depuis une autre machine/téléphone : aucune dépendance au Desktop.

3. Plan de sessions recommandé

Session	Branche	Travail
S0	desktop/architecture	P0
S1	desktop/contracts-core	P1-P2
S-A	desktop/shell	P3
S-B	desktop/daemon	P4
S-C	desktop/workspaces	P5
S-D	desktop/knowledge	P6
S-E	desktop/code-graph	P7
S-F	desktop/graph-viewers	P8
S-G	desktop/harness-adapters	P9
S-H	desktop/packaging	P10
S-I	desktop/integration	P11-P12

Vague recommandée : S0 -> validation -> S1 -> fusion baseline. Puis lancer S-A à S-H en parallèle. S-F peut travailler avec fixtures de graphes pendant S-D/S-E. S-I ne commence les réconciliations qu'une fois les lanes prêtes.

Critères de clôture

- ✓ - Un utilisateur neuf peut installer et utiliser Studi'OS sans ouvrir un terminal.
- ✓ - Le même Dashboard continue à fonctionner sur le web.
- ✓ - Le daemon et ses services sont gérés par l'application et diagnostiquables graphiquement.
- ✓ - Vault et Code Graph sont optionnels par projet et n'empêchent jamais le fonctionnement de base.
- ✓ - Obsidian, OpenCode, Claude Code et autres outils restent des intégrations facultatives.
- ✓ - Knowledge Graph et Code Graph sont visualisables nativement.
- ✓ - Project Graph est une projection traçable des sources, jamais une base opaque.
- ✓ - Les données locales privées ne quittent pas la machine sans action/configuration explicite.
- ✓ - Graphify peut être remplacé derrière CodeGraphProvider si nécessaire.
- ✓ - Installation, upgrade et désinstallation préservent les données utilisateur conformément aux choix documentés.
- ✓ - Les tests existants serveur/dashboard restent verts.

Dogfood conseillé : développer ce chantier depuis Studi'OS lui-même : créer un projet/roadmap dédié, répartir les lanes en sessions parallèles, utiliser le Git Watcher et les tâches existantes, puis tester chaque build Desktop sur une machine/VM comme si elle appartenait à un nouvel utilisateur.